

W11D03 — Dependency Injection yang Proper

ML Engineer Journey | Phase 3 — Docker & Clean Architecture | 20 Mei 2026

1. Motivasi — Kenapa DI Ada

Setelah W11D01–D02 membangun aturan per layer secara konseptual dan implementatif, muncul pertanyaan yang logis: kalau Router butuh Service, dan Service butuh Model — **bagaimana mereka mendapatkan apa yang mereka butuhkan tanpa melanggar aturan layer?**

Tanpa mekanisme yang tepat, jawaban naif adalah: Service mengurus sendiri. Ia membuka file model, memuat ke memori, menyimpan sebagai atributnya. Itu pelanggaran — Service sekarang tahu detail infrastruktur (path file, format serialisasi) yang bukan tanggung jawabnya.

Dependency Injection (DI) adalah solusinya: **sebuah komponen tidak mengurus cara mendapatkan ketergantungannya — ketergantungan itu diberikan dari luar.**

Analogi — Pelayan Restoran

Pelayan tidak pergi ke gudang untuk mengambil notepad-nya sendiri dan membangun sistem komunikasi ke dapur dari nol setiap ada tamu. Restoran yang sehat sudah *menyiapkan semua itu* dan memberikannya ke pelayan saat ia mulai shift. Pelayan tinggal menggunakan — bukan mengurus logistik pengadaannya.

Dalam kode: Router adalah pelayan, Service adalah sistem komunikasi ke dapur, Model adalah bahan masakan. Tidak ada yang mengurus pengadaannya sendiri.

2. Dua Pendekatan Caching — lru_cache vs app.state

Ada dua cara umum untuk memastikan model hanya dimuat sekali:

Aspek	lru_cache	app.state via Lifespan
Di mana state disimpan	Module-level cache (Python process)	Object app.state milik FastAPI instance
Kontrol lifecycle	Python GC + decorator otomatis	Eksplisit — kamu tulis sendiri di lifespan()
Bisa di-reset tanpa restart?	Tidak — butuh cache_clear()	Ya — assign ulang app.state.model
Testability	Butuh cache_clear() antar test	Override langsung via app.state
Cocok untuk	Model stateless, tidak perlu cleanup	Resource yang butuh open/close (DB pool, HTTP client)
Akses di Depends()	Import langsung get_model()	Perlu Request object untuk request.app.state

Untuk model sklearn yang stateless, **lru_cache cukup dan lebih simpel**. `app.state` via Lifespan menjadi relevan di W13 saat database connection pool masuk — karena connection pool *harus* dibuka saat startup dan ditutup saat shutdown.

Keputusannya bukan mana yang lebih 'benar' — tapi mana yang tepat untuk resource yang sedang diurus.

3. Implementasi — Empat File Kunci

model/loader.py

Infrastructure layer. Satu-satunya tempat yang boleh tahu cara memuat model dari disk.

```
from functools import lru_cache
from pathlib import Path
import joblib

MODEL_PATH = Path(__file__).parent.parent / "model.pkl"
# Path(__file__) = lokasi loader.py itu sendiri
# .parent.parent = naik 2 level ke root app/
# Tidak hardcode string – kalau folder pindah, path ikut

@lru_cache(maxsize=1)
def load_model_from_disk():
    if not MODEL_PATH.exists():
        raise FileNotFoundError(f"File model tidak ditemukan: {MODEL_PATH}")
    return joblib.load(MODEL_PATH)
# lru_cache(maxsize=1): panggilan pertama baca file,
# panggilan berikutnya kembalikan dari cache

def get_model(request: Request):
    # Wrapper untuk Depends() – akses via app.state
    if not hasattr(request.app.state, "model"):
        raise RuntimeError("Model belum dimuat ke dalam app.state.")
    return request.app.state.model
```

Guard hasattr penting — kalau lifespan gagal dan ada request masuk, error-nya jelas bukan `AttributeError` yang membingungkan.

main.py — Lifespan dan Eager Loading

Tanpa eager loading, model baru dibaca dari disk saat REQUEST PERTAMA — pengguna pertama menanggung latency load. Dengan lifespan, model sudah siap sebelum server menerima request apapun.

```
from contextlib import asynccontextmanager
from fastapi import FastAPI
import logging

logger = logging.getLogger(__name__)

@asynccontextmanager
async def lifespan(app: FastAPI):
    logger.info("Startup: memuat model ke memori...")
    try:
```

```

app.state.model = load_model_from_disk()
logger.info("Model berhasil dimuat. Server siap.")
except Exception as e:
    logger.critical(f"Gagal memuat model: {e}")
    raise # bukan 'raise e' – pertahankan original traceback
yield # server berjalan menerima request di sini
if hasattr(app.state, "model"):
    del app.state.model
logger.info("Shutdown: cleanup selesai.")

app = FastAPI(title="Risk Scoring API", lifespan=lifespan)

```

raise tanpa argumen mempertahankan traceback asli dari load_model_from_disk(). raise e membuat Python menganggap exception dimulai dari baris itu — traceback asli hilang, debugging jadi lebih sulit.

services/model_service.py — Business Logic Murni

Service layer tidak boleh mengimpor apapun dari FastAPI. Tidak ada Depends(), tidak ada HTTPException, tidak ada Request. Kalau besok service ini dipakai dari CLI atau gRPC, tidak ada yang rusak.

```

import numpy as np
from typing import Any

class ModelService:
    def __init__(self, model: Any):
        self.model = model # Diterima dari luar, tidak diurus sendiri

    def predict(self, features: list[float]) -> dict:
        if not features:
            raise ValueError("Feature list tidak boleh kosong.")
        if len(features) != 5:
            raise ValueError(f"Butuh 5 features, dapat {len(features)}.")
        try:
            arr = np.array(features).reshape(1, -1)
            # reshape(1,-1): ubah [f1..f5] jadi [[f1..f5]] – sklearn butuh 2D
            prediction = self.model.predict(arr)[0]
            probability = self.model.predict_proba(arr)[0].max()
        except Exception as e:
            raise RuntimeError(f"Model gagal inferensi: {e}") from e
        # 'from e' – original traceback tetap utuh di log
        return {
            "prediction": int(prediction),
            "confidence": round(float(probability), 4),
        }
    # TIDAK ADA get_service di sini.
    # TIDAK ADA import FastAPI di sini.

```

Exception language di service: ValueError untuk input tidak valid, RuntimeError untuk kegagalan model. Router yang menerjemahkan ke HTTP status code.

routers/predict.py — Presentation Layer

`Depends()` dan `get_model_service` hidup di sini — bukan di `service`. Router adalah satu-satunya layer yang boleh bicara bahasa HTTP.

```
import logging
from fastapi import APIRouter, Depends, HTTPException, Request
from schemas.prediction import PredictionRequest, PredictionResponse
from services.model_service import ModelService

logger = logging.getLogger(__name__)
router = APIRouter(prefix="/predict", tags=["prediction"])

def get_model_service(request: Request) -> ModelService:
    # Factory function – milik router, bukan service
    # ModelService baru setiap request, tapi model 100MB tidak di-copy
    # Python hanya meneruskan referensi ke objek yang sama di app.state
    return ModelService(model=request.app.state.model)

@router.post("/", response_model=PredictionResponse)
async def predict(
    request: PredictionRequest,
    service: ModelService = Depends(get_model_service),
    # Depends(): FastAPI panggil get_model_service() dulu,
    # masukkan hasilnya sebagai argumen 'service'
):
    try:
        result = service.predict(request.to_feature_list())
        return PredictionResponse(**result)
    except ValueError as e:
        logger.warning(f"Validation error: {e}")
        raise HTTPException(status_code=422, detail=str(e))
    except RuntimeError as e:
        logger.error(f"Model inference error: {e}")
        raise HTTPException(status_code=503, detail=str(e))
    except Exception as e:
        logger.critical(f"Unexpected error: {e}", exc_info=True)
        raise HTTPException(status_code=500, detail="Internal server error")
```

4. Alur Dependency Injection — Urutan Resolusi

FastAPI menyelesaikan dependency dari bawah ke atas, sebelum handler dipanggil:

- ① **`get_model_service(request)`** dipanggil FastAPI secara otomatis
- ② **`request.app.state.model`** diakses — referensi ke objek model di memori
- ③ **`ModelService(model=...)`** dibuat — wrapper ringan, bukan copy model
- ④ **`handler predict()`** dipanggil dengan service yang sudah siap

Kenapa membuat `ModelService` baru setiap request tidak masalah performa? Python tidak menyalin objek — hanya meneruskan alamat memori (referensi). Yang dibuat baru hanyalah wrapper `ModelService`-nya. Model 100MB tetap di satu lokasi di memori, diakses oleh semua request.

5. Aturan Per Layer — Ringkasan

	Router	Service	Repository
Import FastAPI?	✓ Ya	✗ Tidak	✗ Tidak
Import HTTPException?	✓ Ya	✗ Tidak	✗ Tidak
Business logic?	✗ Tidak	✓ Ya	✗ Tidak
Query data langsung?	✗ Tidak	✗ Tidak	✓ Ya
Exception dilempar	HTTPException	ValueError / RuntimeError	LookupError / None
Depends() hidup di?	✓ Di sini	✗ Tidak	✗ Tidak